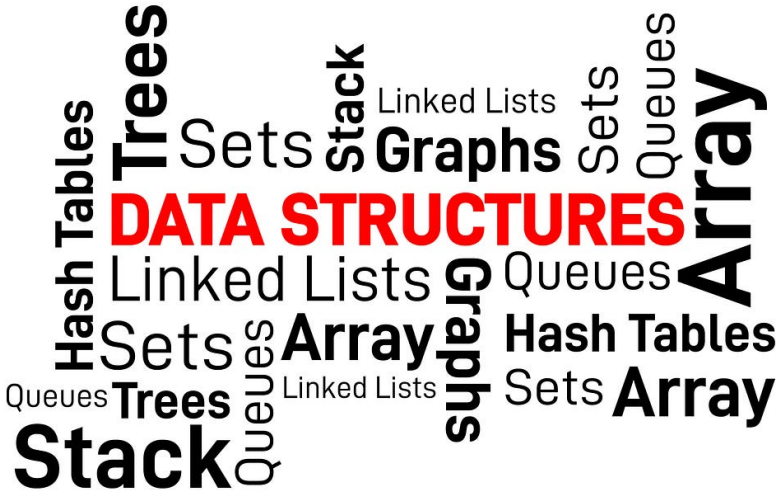


Data Structures

IT623

Dr. Manish Khare

Lecture – 9

Queue

- Queue is an abstract data structure, somewhat similar to Stacks. Unlike stacks, a queue is open at both its ends. One end is always used to insert data (enqueue) and the other is used to remove data (dequeue). Queue follows First-In-First-Out methodology, i.e., the data item stored first will be accessed first.

- **Queue data structure is a collection of similar data items in which insertion and deletion operations are performed based on FIFO principle.**

- A real-world example of queue can be a single-lane one-way road, where the vehicle enters first, exits first. More real-world examples can be seen as queues at the ticket windows and bus-stops.

Queue Representation

- As we now understand that in queue, we access both ends for different reasons. The following diagram given below tries to explain queue representation as data structure −

- As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures. For the sake of simplicity, we shall implement queues using one-dimensional array.

Basic Operations

- Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory. Here we shall try to understand the basic operations associated with queues −
  - **enqueue()** − add (store) an item to the queue.
  - **dequeue()** − remove (access) an item from the queue.
- Few more functions are required to make the above-mentioned queue operation efficient. These are −
  - **peek()** − Gets the element at the front of the queue without removing it.
  - **isfull()** − Checks if the queue is full.
  - **isempty()** − Checks if the queue is empty.
- In queue, we always dequeue (or access) data, pointed by **front** pointer and while enqueing (or storing) data in the queue we take help of **rear** pointer.

Peek()

- This function helps to see the data at the **front** of the queue. The algorithm of peek() function is as follows.
  - **Algorithm**

```
begin procedure peek
  return queue[front]
end procedure
```

- Implementation of peek() function in C/C++ programming language

```
int peek()
{
  return queue[front];
}
```

Isfull()

- As we are using single dimension array to implement queue, we just check for the rear pointer to reach at MAXSIZE to determine that the queue is full. In case we maintain the queue in a circular linked-list, the algorithm will differ. Algorithm of isfull() function

```
begin procedure isfull
  if rear equals to MAXSIZE
          return true
  else
          return false
 endif
end procedure
```

- Implementation of isfull() function in C/C++ programming language

```
bool isfull()
{
    if(rear == MAXSIZE - 1)
 return true;
    else
return false;
}
```

Isempty()

- Algorithm of isempty() function –

If the value of **front** is less than MIN or 0, it tells that the queue is not yet initialized, hence empty.

```
begin procedure isempty
   if front is less than MIN  OR front is greater than rear
      return true
   else
      return false
   endif
end procedure
```

```
bool isempty() {
   if(front < 0 || front > rear)
      return true;
   else
      return false;
}
```

Implementation of Queue

- Queue data structure can be implemented in two ways. They are as follows...
  - **Using Array**
  - **Using Linked List**

- When a queue is implemented using array, that queue can organize only limited number of elements. When a queue is implemented using linked list, that queue can organize unlimited number of elements.

Queue using Array

- The implementation of queue data structure using array is very simple, just define a one dimensional array of specific size and insert or delete the values into that array by using **FIFO (First In First Out) principle** with the help of variables **'front'** and '**rear**'.
- Initially both '**front**' and '**rear**' are set to -1. Whenever, we want to insert a new value into the queue, increment '**rear**' value by one and then insert at that position.
- Whenever we want to delete a value from the queue, then increment 'front' value by one and then display the value at '**front**' position as deleted element.

Queue Operations using Array

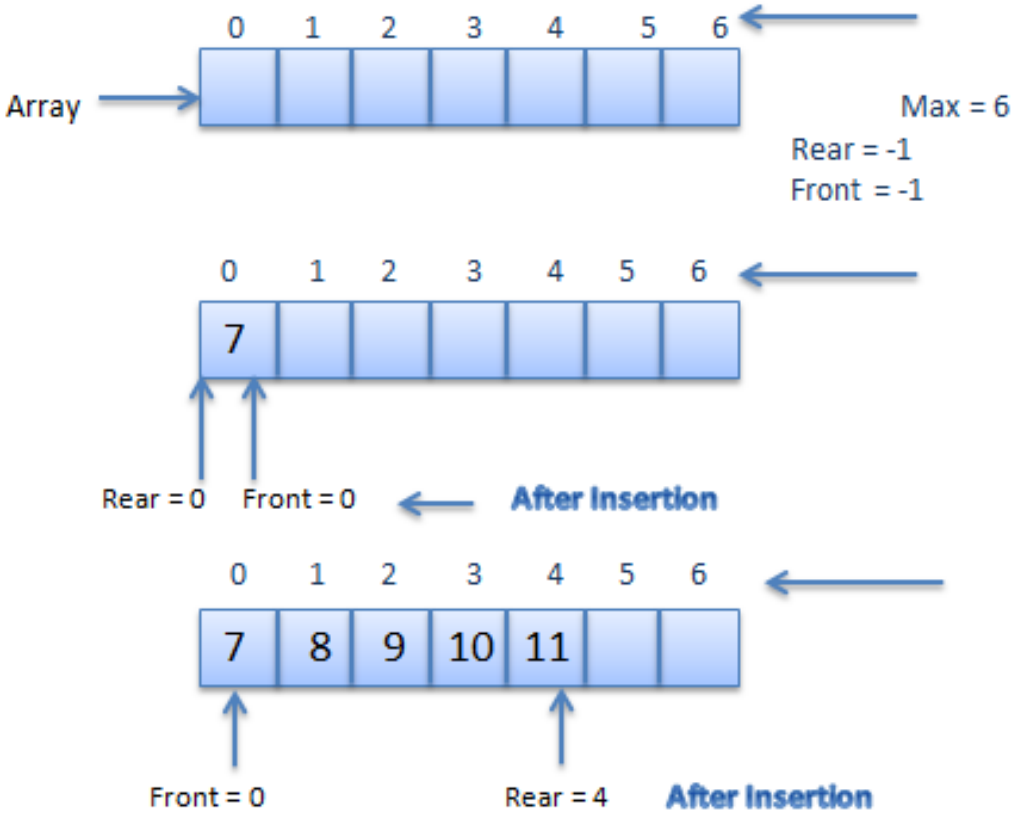
- Queue data structure using array can be implemented as follows...

  Before we implement actual operations, first follow the below steps to create an empty queue.

- **Step 1:** Include all the **header files** which are used in the program and define a constant **'SIZE'** with specific value.
- **Step 2:** Declare all the **user defined functions** which are used in queue implementation.
- **Step 3:** Create a one dimensional array with above defined SIZE (**int queue[SIZE]**)
- **Step 4:** Define two integer variables **'front'** and '**rear**' and initialize both with **'-1'**. (**int front = -1, rear = -1**)
- **Step 5:** Then implement main method by displaying menu of operations list and make suitable function calls to perform operation selected by the user on queue.

enQueue(value) - Inserting value into the queue

- In a queue data structure, enQueue() is a function used to insert a new element into the queue. In a queue, the new element is always inserted at **rear**position. The enQueue() function takes one integer value as parameter and inserts that value into the queue. We can use the following steps to insert an element into the queue...
  - **Step 1:** Check whether **queue** is **FULL**. (**rear == SIZE-1**)
  - **Step 2:** If it is **FULL**, then display **"Queue is FULL!!! Insertion is not possible!!!"** and terminate the function.
  - **Step 3:** If it is **NOT FULL**, then increment **rear** value by one (**rear++**) and set **queue[rear] = value**.


0 1 2 3 4 5 6
Array
Max = 6
Rear = -1
Front = -1
0 1 2 3 4 5 6
7
Rear = 0
Front = 0
After Insertion
0 1 2 3 4 5 6
7 8 9 10 11
Front = 0
Rear = 4
After Insertion

- Step 1: IF REAR = MAX-1
  - Write OVERFLOW
  - Goto step 4
  - [END OF IF]
- Step 2: IF FRONT = -1 and REAR = -1
  - SET FRONT = REAR = 0
  - ELSE
    - SET REAR = REAR + 1
  - [END OF IF]
- Step 3: SET QUEUE[REAR] = NUM
- Step 4: EXIT

deQueue() - Deleting a value from the Queue

- In a queue data structure, deQueue() is a function used to delete an element from the queue. In a queue, the element is always deleted from **front** position. The deQueue() function does not take any value as parameter. We can use the following steps to delete an element from the queue...

  - **Step 1:** Check whether **queue** is **EMPTY**. (**front == rear**)
  - **Step 2:** If it is **EMPTY**, then display **"Queue is EMPTY!!! Deletion is not possible!!!"** and terminate the function.
  - **Step 3:** If it is **NOT EMPTY**, then increment the **front** value by one (**front ++**). Then display **queue[front]** as deleted element. Then check whether both **front** and **rear** are equal (**front == rear**), if it **TRUE**, then set both **front** and **rear** to '**-1**' (**front = rear = -1**).

Queue at any instant

Queue after insertion of a new element

Queue after deletion of an element

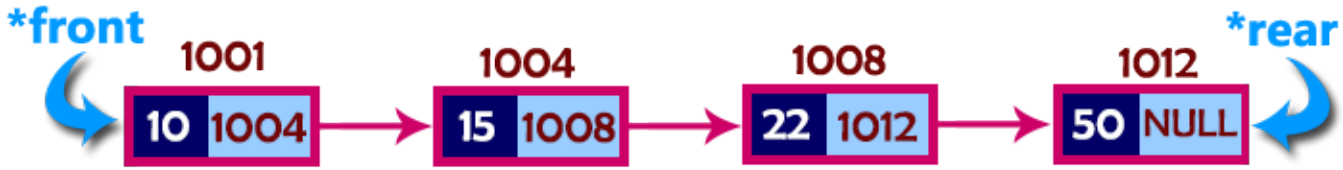
- Step 1: IF FRONT = -1 OR FRONT > REAR
  - Write UNDERFLOW
  - ELSE
    - SET FRONT = FRONT + 1
  - [END OF IF]
- Step 2: EXIT

display() - Displays the elements of a Queue

- We can use the following steps to display the elements of a queue...
  - **Step 1:** Check whether **queue** is **EMPTY**. (**front == rear**)
  - **Step 2:** If it is **EMPTY**, then display **"Queue is EMPTY!!!"** and terminate the function.
  - **Step 3:** If it is **NOT EMPTY**, then define an integer variable '**i**' and set '**i = front+1**'.
  - **Step 4:** Display '**queue[i]**' value and increment '**i**' value by one (**i++**). Repeat the same until '**i**' value is equal to **rear** (**i <= rear**)

Queue using linked list

- The major problem with the queue implemented using array is, It will work for only fixed number of data. That means, the amount of data must be specified in the beginning itself.
- Queue using array is not suitable when we don't know the size of data which we are going to use.
- A queue data structure can be implemented using linked list data structure. The queue which is implemented using linked list can work for unlimited number of values. That means, queue using linked list can work for variable size of data (No need to fix the size at beginning of the implementation).
- The Queue implemented using linked list can organize as many data values as we want.
- In linked list implementation of a queue, the last inserted node is always pointed by '**rear**' and the first node is always pointed by '**front**'.



- In above example, the last inserted node is 50 and it is pointed by '**rear**' and the first inserted node is 10 and it is pointed by '**front**'. The order of elements inserted is 10, 15, 22 and 50.

Queue Operations using Linked List

- To implement queue using linked list, we need to set the following things before implementing actual operations.

  - **Step 1:** Include all the **header files** which are used in the program. And declare all the **user defined functions**.
  - **Step 2:** Define a '**Node**' structure with two members **data** and **next**.
  - **Step 3:** Define two **Node** pointers '**front**' and '**rear**' and set both to **NULL**.
  - **Step 4:** Implement the **main** method by displaying Menu of list of operations and make suitable function calls in the **main** method to perform user selected operation.

enQueue(value) - Inserting an element into the Queue

- We can use the following steps to insert a new node into the queue...
  - **Step 1:** Create a **newNode** with given value and set '**newNode → next**' to **NULL**.
  - **Step 2:** Check whether queue is **Empty** (**rear** == **NULL**)
  - **Step 3:** If it is **Empty** then, set **front** = **newNode** and **rear** = **newNode**.
  - **Step 4:** If it is **Not Empty** then, set **rear → next** = **newNode** and **rear** = **newNode**.

Linked Queue

Linked Queue after inserting a new node

deQueue() - Deleting an Element from Queue

- We can use the following steps to delete a node from the queue...
  - **Step 1:** Check whether **queue** is **Empty** (**front == NULL**).
  - **Step 2:** If it is **Empty**, then display **"Queue is Empty!!! Deletion is not possible!!!"** and terminate from the function
  - **Step 3:** If it is **Not Empty** then, define a Node pointer '**temp**' and set it to '**front**'.
  - **Step 4:** Then set '**front = front → next**' and delete '**temp**' (**free(temp)**).

Linked Queue

Linked Queue after deletion of an element

display() - Displaying the elements of Queue

- We can use the following steps to display the elements (nodes) of a queue...
- **Step 1:** Check whether queue is **Empty** (**front** == **NULL**).
- **Step 2:** If it is **Empty** then, display **'Queue is Empty!!!'** and terminate the function.
- **Step 3:** If it is **Not Empty** then, define a Node pointer **'temp'** and initialize with **front**.
- **Step 4:** Display '**temp** → **data** --->' and move it to the next node. Repeat the same until '**temp**' reaches to '**rear**' (**temp** → **next** != **NULL**).
- **Step 4:** Finally! Display '**temp** → **data** ---> **NULL**'.

Type of Queue

- A queue data structure can be classified into the following types:
  - 1. Circular Queue
  - 2. Dequeue
  - 3. Priority Queue
  - 4. Multiple Queue